

Study Mode - SSM big-picture introduction

**lec17.pdf**

PDF

**lec16.pdf**

PDF

Interactive SSM Tutor

You are my interactive tutor for modern State Space Models (SSMs) in deep learning — including their mathematical foundations, continuous-time vs discrete-time views, stability, long-range dependencies, discretization, and selective input-dependent gating as used in architectures such as Mamba.

All explanations must be based on the lecture slides I provide (lec16.pdf and lec17.pdf), including their diagrams, equations, and intuitions. At every step, connect what you say to content explicitly present in those slides.

This tutoring session should be deeply interactive. In every reply:

- Keep answers focused and concise (3–6 short paragraphs).
- Do not move on to the next topic unless I say so.
- Always end with a specific question (concept check, clarification, or small exercise).
- Adapt dynamically to my answers:
 - If I show misunderstanding, give a short explanation and ask a clarifying question.
 - If my answer is correct and confident, increase the complexity of the next question.
 - If I am confused at a basic level, go back to fundamentals.

★ Overall Story I Want to Learn

Follow this narrative in order unless I say otherwise:

1. Big-picture motivation:
 - Why SSMs are introduced as an alternative to attention for long-range dependencies (lec16).
 - The efficiency vs. efficacy tradeoff and the desire to keep memory of the past while being stable.
 - Role of linear time-invariant (LTI) systems and why structure matters.
2. Foundations of SSMs:
 - The core equations $h_{t+1} = A h_t + B u_t$, $y_t = C h_t$.
 - Interpret “history captured in the state” and why dimension N matters (lec16, p.1).
 - How recurrence and convolution views are equivalent (lec17, p.5–10).
3. Stability & eigenvalues:
 - How eigenvalues of A determine whether memory explodes, decays, or persists (lec16, p.2).
 - Why long-range memory requires eigenvalues near the unit circle.
 - Real vs complex eigenvalues and the motivation for HIPPO-style initialization (lec16, p.3).
4. Continuous-time SSMs $\rightarrow$ discretization:
 - The CT system $\dot{x}(t) = A x(t) + B u(t)$, $y(t) = C x(t)$ (lec17, p.5).
 - Step-size Δt as a degree of freedom and its effect on stability and memory (lec17, p.6–7).
 - Why $e^{A\Delta t}$ matters and what small vs. large Δt does.
5. Input-dependent selective gating (Mamba):
 - Why changing Δt dynamically allows the model to modulate “how much of the past” to keep (lec17, p.7).
 - The selective-scan mechanism and how it combines recurrence & convolution views efficiently (lec17, p.10).
 - How this enables long-range reasoning with linear-time inference.
6. Putting it all together:

- How SSMs unify recurrence, convolution, and long-range memory.
 - How stability, eigenvalues, discretization, and gating combine in modern SSM architecture design.
-

Step 1: First Response Behavior

Your first response should include:

1. A two-paragraph big-picture intro, summarizing:
 - Why SSMs aim to efficiently model long-range dependencies while keeping stability under control.
 - The idea that the state must encode all relevant past information, but eigenvalues of A determine how long memory persists or explodes.
 - The CT $\rightarrow$ DT viewpoint and the importance of discretization ($e^{A\Delta t}$).
 - The idea of selective input-dependent gating as in Mamba.
2. A roadmap of the main topics (the 6 bullets above), explained as a coherent storyline following the slides.
3. End your message with one short question to check my high-level understanding (e.g., "Why does the stability of A matter for memory length?").

After I answer, move to Topic 1 unless I say otherwise.

Step 2: Teaching Each Topic — Four Layers

For each topic in the roadmap, structure your response as:

1. Intuition (linked to slides)
 - Use 1–2 paragraphs to relate the concept to the lecture content (e.g., stability diagrams on lec16 p.2, CT curves on lec17 p.6).
 - Always link the intuition to SSM goals: efficient long-

range memory, avoiding explosion, enabling useful recurrence.

2. Key Formal Setup

- Introduce just the necessary notation.
- Use small equations like those on the slides (e.g., discretization formula, eigenvalue conditions).
- Describe verbally what each piece means.

3. Tiny Example

- Provide a tiny numeric or qualitative example (e.g., a 1D CT system with $a < 0$, effect of Δt , or a matrix with eigenvalues 0.5 and 1.02 and what that implies).
- Make sure it corresponds to examples shown in the notes when possible.

4. Understanding Check

- Ask 1–2 short questions: one conceptual, optionally one computational.
- Then stop and wait for my answer.

End each message by reminding me what the next topic will be only if I say I'm ready to move on.

Step 3: Final Wrap-Up

When I say I'm ready for a review:

Provide a compact summary that connects:

- Stability & eigenvalues → memory behavior.
- CT → DT conversion → practical SSM layers.
- Selective gating → adaptive long-range reasoning.
- How these components align with diagrams and equations in lec16 & lec17.

The summary should read like a cohesive, lecture-aligned storyline.

Final Instruction

Begin now with Step 1: give the two-paragraph big-picture intro, the roadmap, and a final comprehension question — all grounded in the slide content.

Here is your **Step 1** response — big-picture introduction, roadmap, and a final comprehension question — all grounded in *lec16* and *lec17*.

Big-Picture Introduction (2 paragraphs)

Modern **State Space Models (SSMs)** are motivated by a simple but fundamental challenge shown in *lec16 p.1*: **how can a model keep useful information from the distant past without exploding or forgetting too quickly?** Transformers store all past tokens in RAM-like attention, which is powerful but expensive. SSMs instead compress all past information into a hidden **state** h_t , updated by a linear recurrence $h_{t+1} = Ah_t + Bu_t$. But this introduces a delicate balance: the model must retain long-range dependencies efficiently *and* remain stable over thousands of steps. The structure of the matrix A — particularly its eigenvalues (as shown on *lec16 p.2*) — determines whether memory decays too fast, persists appropriately, or explodes.

The second major theme comes from *lec17*: viewing SSMs in **continuous time** $\dot{x}(t) = Ax(t) + Bu(t)$ and then discretizing them with a step size Δt . The key object is $e^{A\Delta t}$, shown repeatedly in the derivations on *lec17 p.5–7*. Changing Δt changes how fast the system forgets or responds to new input. This provides a powerful idea: **make Δt input-dependent**, giving the model selective gating ability — the core of Mamba's "selective scan." When the input suggests that history should matter, choose small Δt ; when the model should forget, choose large Δt . This yields long-range reasoning at linear-time inference cost.

Roadmap of What You Will Learn

Following the slides chronologically, here is the storyline we will go through:

1. Big-picture motivation (lec16)

- Why we want long-range memory without the cost of attention.
- Efficiency vs. efficacy tradeoff and why the entire past must fit into the state.

2. Foundations of SSMs (lec16 → lec17)

- The core equations $h_{t+1} = Ah_t + Bu_t$, $y_t = Ch_t$.
- Why the state dimension N determines the model's "memory capacity."
- How recurrence and convolution are mathematically equivalent.

3. Stability & eigenvalues (lec16 p.2–3)

- $|\lambda| < 1 \rightarrow$ memory dies out; $|\lambda| > 1 \rightarrow$ unstable explosion; $|\lambda| \approx 1 \rightarrow$ long memory.
- Complex eigenvalues & HIPPO initialization for stable long-range behavior.

4. Continuous-time SSMs & discretization (lec17 p.5–7)

- CT dynamics $\dot{x} = Ax + Bu$.
- Discretization via $e^{A\Delta t}$ and the critical role of Δt .
- How step size affects forgetting vs. remembering.

5. Selective input-dependent gating (lec17 p.7–10)

- How varying Δt based on the input gives adaptive memory.
- How Mamba's selective scan merges recurrence and convolution efficiently.

6. Putting it all together

- A unified picture: stability + CT→DT + selective gating produces modern SSMs capable of long-range reasoning with linear-time inference.

Comprehension Check (1 question)

In the discrete recurrence $h_{t+1} = Ah_t + Bu_t$, why does the stability of the matrix A directly control how long the model can remember past inputs?

(Answer this, and then I'll move on to Topic 1.)

If A is instable, ie eigenvalues too large / too small then the hidden state will either blow up / go to zero

Great — that's exactly the core idea from *lec16 p.2*. Since $h_t = A^t h_0 + \sum_{k < t} A^{t-k-1} B u_k$, the powers of A determine how past information is amplified or suppressed. If $|\lambda| > 1$, terms grow; if $|\lambda| < 1$, they vanish; only $|\lambda| \approx 1$ preserves memory over long horizons.

Let's begin **Topic 1: Big-Picture Motivation**.

Topic 1 — Big-Picture Motivation (lec16)

1. Intuition (linked to slides)

On *lec16 p.1*, the key question is written directly:

"How to support long-range dependencies?"

The slide emphasizes that *all* information about the past must be compressed into the hidden state h_t , because SSMs do not store the full history like attention does. This leads to the core tension: if the state is too small or too unstable, long-range dependencies disappear; if the state is large or A is unstable, computation becomes expensive or numerically problematic.

The slides also highlight the **efficiency vs. efficacy tradeoff**. Larger state dimension N gives more expressivity ("more memory/history capacity"), but increases compute. At the same time, we want A to be structured — typically from LTI systems — so we can implement the model efficiently. This power/efficiency tension is the motivation behind the entire SSM line of work, culminating in layers like Mamba.

2. Key Formal Setup

The foundational model is:

$$h_{t+1} = A h_t + B u_t, \quad y_t = C h_t.$$

- u_t : input token embedding.
- h_t : compressed representation of *all past inputs*.
- A, B, C : learned parameters, often constrained or initialized with special structure.

The whole idea is that h_t must carry forward everything relevant from $(u_0, \dots, u_t)$ *without* exploding or collapsing. This setup is what sets up the importance of eigenvalue placement and stability, which we tackle in Topic 3.

3. Tiny Example

Suppose $A = 0.99$. Then each step multiplies the memory by $\sim 0.99 \rightarrow$ slow decay $\rightarrow$ can remember hundreds of steps.

If $A = 0.1$, memory decays almost instantly.

If $A = 1.01$, history grows exponentially $\rightarrow$ unstable.

This simple scalar picture is exactly what the eigenvalue diagram on *lec16 p.2* is capturing.

4. Understanding Check

Concept check: Why does increasing the state dimension N (the size of h_t) tend to improve the model's ability to store more complex or longer-term information?

(Once you answer, we'll move to Topic 2: Foundations of SSMs.)

if you increase N then you fundamentally just have more bits of storage to store information

Exactly — that's the core point on *lec16 p.1*: the entire past must be "fit" into the state h_t , so increasing N increases the model's **capacity to store richer or longer histories**. Larger N means more dimensions to encode slower modes, oscillatory modes, multiple timescales, etc.

Great — let's move to **Topic 2: Foundations of SSMs**.

Topic 2 — Foundations of SSMs (lec16 $\rightarrow$ lec17)

1. Intuition (linked to slides)

On *lec16 p.1*, the core equations

$$h_{t+1} = Ah_t + Bu_t, \quad y_t = Ch_t$$

are highlighted as the basic recurrence for state-space models. The slide explicitly notes that *all past information must be captured in the state*, reinforcing why designing A well is essential.

Now, on *lec17 p.5*, the notes introduce the idea that SSMs have *two equivalent views*:

- **Recurrent (state-based) view:** update h_t step by step during *inference time*.
- **Convolutional view:** treat the whole sequence as passed through a long 1D convolution during *training time*.

This equivalence is crucial because the recurrent form is efficient for long sequences (linear time), while the convolutional view allows parallel training. The slides sketch this duality by showing the CT system $\rightarrow$ DT system $\rightarrow$ convolution kernels.

2. Key Formal Setup

Discrete-time system:

$$h_{t+1} = Ah_t + Bu_t, \quad y_t = Ch_t.$$

If you unroll h_{t+1} repeatedly:

$$h_t = A^t h_0 + \sum_{k=0}^{t-1} A^{t-k-1} B u_k.$$

Plugging this into $y_t = Ch_t$ gives:

$$y_t = \sum_{k=0}^{t-1} C A^{t-k-1} B u_k.$$

This is precisely a **convolution** with kernel

$$K_{t-k} = C A^{t-k-1} B.$$

This matches the diagrams on *lec17 p.5–10*, where the DT recurrence is shown to generate a convolution over time.

3. Tiny Example

Let

$$A = 0.9, \quad B = 1, \quad C = 1.$$

Then the kernel becomes:

$$K_0 = 1, \quad K_1 = 0.9, \quad K_2 = 0.9^2, \quad K_3 = 0.9^3, \dots$$

This is an exponentially decaying convolution kernel — exactly the shape shown in the colored decay curves on *lec17 p.5–6*.

If A had complex eigenvalues (like in HIPPO initialization on *lec16 p.3*), the kernel would oscillate while decaying.

4. Understanding Check

Concept check: Why does the equivalence between the recurrent view and the convolutional view make SSMs efficient during training but still linear-time during inference?

(Answer this, and then we'll proceed to Topic 3: Stability & eigenvalues.)

Im not too sure. Is it because FFT can compute convolutions efficiently?

Great instinct — yes, FFTs *are* one route, but let's make the connection fully clear using exactly what the slides emphasize.

✓ **Clarification (based on the slides)**

On *lec17 p.5–10*, the key idea is:

- The **recurrent view**

$$h_{t+1} = Ah_t + Bu_t$$

is inherently sequential → perfect for **inference**, where you only need to process one token at a time.

- The **convolution view**

$$y_t = \sum_{k=0}^{t-1} K_{t-k} u_k, \quad K_{t-k} = CA^{t-k-1}B$$

is a big 1D convolution over the *whole* sequence → perfect for **training**, because training can be parallelized across all time indices.

The slide diagrams show exactly this duality:

recurrent = step-by-step, convolution = global + parallel.

FFT is one possible acceleration when the kernel is long, but the deeper point — the one the notes want you to see — is:

Training can use convolution because all time steps are known, so we can compute outputs in parallel.

Inference must use recurrence because tokens arrive one at a time.

So the dual views give:

- **Training:** fast parallel convolution (via FFT or direct kernel application).
- **Inference:** fast sequential recurrence ($O(1)$ per token → $O(T)$ total).

The slides emphasize that this duality is why SSMs offer a promising alternative to attention: **parallel training + efficient inference.**

★ Understanding Check

Here's a sharper question to test comprehension:

If you only had the recurrent view (no convolution view), why would training be much slower on long sequences?

(Answer this, and then we'll move to Topic 3: Stability & eigenvalues.)

I see. Then in this case training would not be parallelizable.

Exactly — that's the key insight the slides are pushing toward.

If you only have the recurrence $h_{t+1} = Ah_t + Bu_t$, then training would require processing the entire sequence **step by step**, making it inherently sequential and slow. The convolution view unlocks full parallelism.

Great — you're ready for **Topic 3: Stability & Eigenvalues**.

Topic 3 — Stability & Eigenvalues (lec16 p.2–3)

1. Intuition (linked to slides)

On *lec16 p.2*, the notes draw a diagram of eigenvalues $\lambda_1, \dots, \lambda_N$ of the matrix A and label three regimes:

- $|\lambda| > 1$: **system explodes**
- $|\lambda| < 1$: **history dies out quickly**
- $|\lambda| = 1$: **critical damping / persistent memory**

This directly expresses why SSMs must place eigenvalues carefully: they “control the fate of memory.”

If eigenvalues are too small, long-range dependencies vanish; too large, and the system becomes numerically unstable.

On *lec16 p.3*, the slides note that if A has **real** eigenvalues and we require $|\lambda| = 1$, there are only trivial choices (± 1). That's not enough variety to encode rich memory patterns. The workaround is **complex eigenvalues** and structured initializations like **HIPPO**, which produce oscillatory-but-stable modes capable of expressing more nuanced long-range behavior.

2. Key Formal Setup

If

$$h_{t+1} = Ah_t,$$

then

$$h_t = A^t h_0.$$

If A is diagonalizable with eigenvalues λ_i , then

$$A^t v_i = \lambda_i^t v_i.$$

Thus the contribution of the i -th mode behaves like λ_i^t .

This is exactly why the slide's diagram plots $|\lambda|$ against behavior:

- $|\lambda| < 1$: exponential decay.
 - $|\lambda| = 1$: stable oscillations or persistence.
 - $|\lambda| > 1$: exponential explosion.
-

3. Tiny Example

Consider a 2D system with eigenvalues:

1. $\lambda_1 = 0.95 \rightarrow$ decays slowly
2. $\lambda_2 = e^{i0.3} \rightarrow$ oscillates without shrinking

This system can encode both:

- a slowly fading memory component, and
- a periodic/oscillatory component.

This mirrors exactly the motivation behind complex eigenvalues in the slides on *lec16 p.3*: richer dynamics near the unit circle.

4. Understanding Check

Concept check:

Why do SSMs benefit from placing eigenvalues *near but slightly inside* the unit circle instead of exactly on it?

(Once you answer, we'll move to Topic 4: continuous-time SSMs → discretization.)

if it were only inside the unit circle we would have a limited amount of eigenvalues to choose from

Ah — good catch: this is actually the *opposite* of what the slides are emphasizing. Let's unpack it gently.

✓ Quick Explanation (connecting to *lec16 p.2–3*)

The issue is **not** that we'd have too few eigenvalues to choose from if they were inside the unit circle. In fact, the entire *interior* of the unit disk is available — an infinite continuum of choices.

Instead, the key reason SSMs place eigenvalues **just inside** the unit circle is:

If eigenvalues are exactly on the unit circle, the system becomes marginally stable and extremely sensitive to noise or parameter updates.

A tiny training update can push λ outside the unit circle → instability.

A tiny floating-point perturbation can accumulate → drift.

This is why *lec16 p.2* labels $|\lambda| = 1$ as a knife-edge boundary.

So SSMs place eigenvalues like:

$$|\lambda| = 1 - \varepsilon, \quad \varepsilon \ll 1$$

so that:

- memory lasts a long time (slow decay),
- but is **guaranteed** to decay eventually → stable,
- and training does not accidentally push the model into explosive behavior.

On *lec16 p.3*, the slides also emphasize that **complex eigenvalues** allow oscillatory (rich) memory even when $|\lambda| < 1$, which solves the “too simple if real” problem.

✓ Clarifying Question

Why does placing eigenvalues exactly on the unit circle make the model *numerically fragile* during training or long-sequence inference?

(Answer this, and then we'll move on to Topic 4: CT → DT discretization.)

because optimizer steps may push the eigenvalues out of the unit circle leading to instability

Exactly — that's the core reason emphasized on *lec16 p.2*:

eigenvalues on the unit circle create **marginal stability**, so even tiny parameter updates or floating-point noise can push the system into the explosive region $|\lambda| > 1$. Keeping them **slightly inside** stabilizes training while preserving long memory.

You're ready for **Topic 4: Continuous-Time SSMs → Discretization**.

Topic 4 — CT SSMs and Discretization (lec17 p.5–7)

1. Intuition (linked to slides)

On *lec17 p.5*, the slides introduce the **continuous-time (CT)** system:

$$\dot{x}(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t).$$

The lecture highlights that CT systems naturally describe **smooth dynamics** and allow us to reason about timescales independently of the discrete step size. The key insight appears on *lec17 p.6–7*: **discretization introduces the factor $e^{A\Delta t}$** , meaning the *effective* discrete dynamics depend both on the matrix A and the chosen step size Δt .

This is where SSMs gain a new degree of freedom. Changing Δt effectively changes how “fast” the system forgets or responds. Smaller $\Delta t \rightarrow$ stay close to identity $\rightarrow$ long memory; larger $\Delta t \rightarrow$ strong decay $\rightarrow$ short memory. The plots on *lec17 p.6* show how increasing Δt pushes discrete eigenvalues toward 0 (stronger damping).

2. Key Formal Setup

If the CT system is:

$$\dot{x}(t) = Ax(t) + Bu(t),$$

then discretizing with step size Δt yields:

$$x_{t+1} = \bar{A}x_t + \bar{B}u_t,$$

where

$$\bar{A} = e^{A\Delta t}, \quad \bar{B} = \left(\int_0^{\Delta t} e^{A\tau} d\tau \right) B.$$

Thus the **effective** eigenvalues in discrete time are:

$$\lambda_{DT} = e^{\lambda_{CT}\Delta t}.$$

On the slide figure:

- When Δt increases, these eigenvalues shrink toward 0 (strong decay).
- When Δt is small, eigenvalues remain close to 1 (long memory).

3. Tiny Example

Let a CT eigenvalue be:

$$\lambda_{CT} = -0.1.$$

Then for different step sizes:

- $\Delta t = 0.01$:

$$e^{-0.1 \cdot 0.01} \approx 0.999$$

→ extremely slow decay → preserves long-range memory.

- $\Delta t = 1.0$:

$$e^{-0.1 \cdot 1} \approx 0.904$$

→ noticeable decay each step → short memory.

This mirrors the curves on *lec17 p.6*, where varying Δt tilts the decay rate.

4. Understanding Check

A conceptual and a computational question:

1. Concept:

Why does decreasing Δt make the discrete system behave *more stably* and preserve memory longer?

2. Compute:

If a CT eigenvalue is $\lambda_{CT} = -2$ and $\Delta t = 0.5$, what is the corresponding DT eigenvalue $e^{A\Delta t}$?

(After you answer, we'll move to Topic 5: selective input-dependent gating, the core of Mamba.)

1. if delta T is smaller then the exponent is smaller and thus the effective eigenvalue is larger (since the base is < 1)
2. 1/e

Excellent — both answers are correct and match exactly what *lec17 p.6–7* is illustrating.

1. Concept:

Yes — when Δt is small, the exponent $\lambda_{CT}\Delta t$ becomes closer to 0, so

$$e^{\lambda_{CT}\Delta t} \approx 1 - |\lambda_{CT}|\Delta t,$$

meaning the discrete eigenvalue sits near 1 $\rightarrow$ long memory + numerical stability.

2. Compute:

With $\lambda_{CT} = -2$ and $\Delta t = 0.5$:

$$e^{-2 \cdot 0.5} = e^{-1} = \frac{1}{e}.$$

Perfect.

You're ready for **Topic 5: Selective input-dependent gating (Mamba)**.

Topic 5 — Selective Input-Dependent Gating (lec17 p.7–10)

1. Intuition (linked to the slides)

On *lec17 p.7*, the slides present the core idea:

What if Δt is not fixed, but *depends on the input*?

Since

$$\bar{A} = e^{A\Delta t},$$

modulating Δt lets the model control how much of the past to keep **dynamically and token-by-token**:

- **Small Δt** $\rightarrow \bar{A}$ close to identity $\rightarrow$ keep past information (long memory).
- **Large Δt** $\rightarrow \bar{A}$ shrinks $\rightarrow$ forget rapidly.

This acts exactly like a **gating mechanism**, but derived from continuous-time dynamics rather than manually designed gates (LSTM/GRU).

The slide diagrams illustrate this effect by showing how varying Δt changes the decay curve of the discretized system.

On *lec17 p.10*, the “**selective scan**” construction shows how Mamba uses this idea to combine the efficient recurrence with a convolution-like parallel view — maintaining linear-time inference but still adapting memory depth to the input.

2. Key Formal Setup

In selective SSMs (like Mamba), Δt becomes a function of the input:

$$\Delta t = f(u_t),$$

so the system updates via:

$$h_{t+1} = e^{A\Delta t(u_t)} h_t + \bar{B}(u_t) u_t.$$

Thus the “gate” is:

$$g_t = e^{A\Delta t(u_t)}.$$

This gate scales each memory mode differently depending on the token. Importantly, because the eigenvalues of A encode different timescales, the gate gives the model *input-dependent control* over those timescales.

3. Tiny Example

Suppose the CT eigenvalue is $\lambda_{CT} = -0.1$.

- For a token that should be remembered:

$$\Delta t = 0.01 \Rightarrow e^{-0.001} \approx 0.999 \quad (\text{almost full memory retention})$$

- For a token that should trigger forgetting:

$$\Delta t = 0.5 \Rightarrow e^{-0.05} \approx 0.951 \quad (\text{much stronger decay})$$

This simple difference in input-dependent Δt produces dramatically different memory behaviors — exactly what *lec17 p.7–10* shows.

4. Understanding Check

1. Concept:

Why is modulating $\Delta t(u_t)$ *more expressive* than multiplying h_t by a learned scalar gate (like in GRUs/LSTMs)?

2. Mechanics:

If $\lambda_{CT} = -0.2$, $A\Delta t = -0.2 \cdot 3$, what discrete eigenvalue do you get?

(After you answer, we'll proceed to the final integrative topic.)

1. im not sure
2. $-0.2^{-0.6}$

Nice, thanks for being honest about #1 — let's clear both up quickly.

For (2), remember the CT→DT rule from the slides:

$$\lambda_{DT} = e^{\lambda_{CT}\Delta t}.$$

Here $\lambda_{CT} = -0.2$ and $\Delta t = 3$, so

$$\lambda_{DT} = e^{-0.2 \cdot 3} = e^{-0.6}.$$

That's a *number* (≈ 0.55), not " $-0.2^{-0.6}$ ". The important part: it's **less than 1 but not tiny**, so memory decays, but not instantly — exactly like the decay curves vs. Δt on lec17 where larger Δt pushes eigenvalues toward 0.

For **(1)**, a plain learned scalar gate (like in a GRU/LSTM) does:

$$h_{t+1} = g_t \odot h_t + (\text{input term}),$$

where g_t just scales each dimension (maybe elementwise) with no built-in notion of timescales. In contrast, changing $\Delta t(u_t)$ changes the **matrix exponential** $e^{A\Delta t(u_t)}$. Because A has a spectrum of eigenvalues encoding different time scales (slow modes, fast modes, oscillatory modes), modulating Δt **rescales all those modes in a structured way**: slow modes stay near 1, fast modes shrink more, complex modes rotate+shrink, etc. So the "gate" is not just "multiply everything by g_t ", it's "apply a CT-aware, timescale-aware transformation" — exactly the selective behavior the slides show in the CT→DT + selective-scan diagrams.

This is why Mamba-style selective SSMs are more expressive: the input doesn't just decide "remember vs forget," it decides **how each dynamical mode of the system evolves** at that step, via $\Delta t(u_t)$ and the shared matrix A .

Quick check

Imagine a particular eigenmode with CT eigenvalue $\lambda_{CT} < 0$.

If the model makes $\Delta t(u_t)$ **very large** at some token (e.g. because that token is unimportant), what happens qualitatively to that mode's discrete eigenvalue $e^{\lambda_{CT}\Delta t(u_t)}$? And how does that affect the influence of *earlier* tokens stored in that mode?

the eigenvalue will decrease and thus earlier tokens will be forgotten more

Exactly — beautifully put.

When $\Delta t(u_t)$ becomes large, the discrete eigenvalue

$$e^{\lambda_{CT}\Delta t(u_t)}$$

shrinks toward 0 (as shown in the decay curves on *lec17 p.6*). A small eigenvalue kills that mode's contribution from earlier tokens, so **the model selectively forgets past information in a principled, dynamics-aware way**.

You're now ready for **Topic 6: Putting It All Together** — the final integrative layer.

Topic 6 — Putting It All Together

1. Intuition (slides lec16 + lec17)

The whole SSM story stitches together the ideas shown across the two lectures:

- From *lec16*:
The hidden state must encode all relevant history, and this requires careful control over **stability** and **eigenvalues** of A . The entire “memory spectrum” lives in the eigenvalues: some modes decay slowly, some quickly, some oscillate.
- From *lec17*:
Thinking in **continuous time** gives a clean picture of timescales.
Discretization with Δt determines how those timescales manifest in discrete recurrence. Varying Δt lets the model adjust memory depth dynamically.
- Combined in modern SSMs (like Mamba):
The model uses the CT→DT structure and the selective-scan derivation (*lec17 p.10*) to run in **linear time**, while dynamically deciding how much history to keep or forget at each token.

2. Key Formal Picture

The system at each time step is:

$$h_{t+1} = e^{A \Delta t(u_t)} h_t + \bar{B}(u_t) u_t.$$

Here:

- A : holds the "modes of memory" (timescales, oscillations, decay rates).
- $\Delta t(u_t)$: input-dependent memory controller.
- Ch_t : reads out state for predictions.

During training, the same dynamics appear as **convolutions** with kernels

$$K_\tau = CA^\tau B,$$

aligning with the convolution diagrams on *lec17 p.5–7*. This dual form gives us parallel training + efficient inference.

3. Tiny Example (full story)

Imagine a CT eigenvalue $\lambda_{CT} = -0.05$ (very slow decay).

If the model encounters an important token, it outputs a small $\Delta t(u_t)$, so

$$e^{-0.05 \cdot \Delta t} \approx 1$$

→ strong memory retention.

A noisy or irrelevant token yields large $\Delta t(u_t)$:

$$e^{-0.05 \cdot \Delta t} \ll 1$$

→ fast forgetting.

This is precisely the *input-dependent timescale modulation* shown in the selective-scan slide.

4. Understanding Check

To confirm deep understanding, answer this:

Why is it powerful that modern SSMs separate (1) the fixed dynamical modes encoded in A from (2) the input-dependent modulation via $\Delta t(u_t)$? How does this separation combine stability, expressivity, and efficiency?

Once you answer, I can give a compact final wrap-up if you want.

I'm not too sure to be honest. Here is my understanding: the fixed dynamical modes determines the memory spectrum of the hidden state and the input-dependent modulation changes this memory spectrum to remember certain channels more of less based on the input. the modes ensure that A is stable, and the modulation enhances expressivity

This is **very close**, and you've captured the two essential ingredients. Let me refine it using the structure emphasized in *lec16* and *lec17*, so the full picture "clicks."

✓ The Clean Version (building on your answer)

1. Fixed modes in A = stable, reusable memory basis

The matrix A encodes a **set of stable dynamical modes**: slow modes, fast modes, oscillatory modes.

These modes are:

- **carefully initialized** (e.g., HIPPO) so eigenvalues lie just inside the unit circle → *lec16 p.2–3*.
- **shared across all time steps** → the system is predictable, smooth, and numerically stable.
- **rich enough to represent many timescales** → the "memory spectrum" you mentioned.

This is the backbone of the SSM: it gives the model a **fixed, well-behaved memory physics**.

2. Input-dependent modulation via $\Delta t(u_t) = \text{adaptive timescale control}$

Changing $\Delta t(u_t)$ affects $e^{A\Delta t(u_t)}$, which means:

- Each eigenmode is scaled according to its CT decay rate.
- Small Δt preserves slow modes $\rightarrow$ long memory.
- Large Δt damps modes $\rightarrow$ selective forgetting.
- The system becomes **input-aware**: the model decides *which parts of the memory spectrum to emphasize* at each token.

This gives the SSM **expressivity** without touching the stability-critical matrix A .

3. Why separating “A” and “modulation” is powerful

This separation accomplishes three things the slides highlight implicitly:

Stability (from A)

A stays fixed and stable $\rightarrow$ eigenvalues remain inside the unit circle $\rightarrow$ long sequences don't blow up.

Training doesn't destabilize the system because the modulation only changes Δt , not A 's eigenvalues.

Expressivity (from $\Delta t(u_t)$)

The model can *re-weight* all timescales dynamically:

- emphasize slow modes for important context,
- kill fast modes when noise appears,
- adjust oscillatory components as needed.

This is more expressive than any scalar gate because $e^{A\Delta t}$ reshapes the full spectrum.

Efficiency (from recurrence + convolution dual view)

- Convolutional form $\rightarrow$ parallel training (*lec17 p.5–7*).
- Recurrent form $\rightarrow$ linear-time inference (*lec17 p.10*).

Modulation doesn't break this dual structure.

So the split lets us have **all three at once**:

stable + expressive + efficient — exactly the design target of modern SSM architectures.

✓ Now for your Understanding Check

Here's the final conceptual question for this topic:

If we allowed the model to update the matrix A itself at every time step instead of only modulating Δt , what major stability and computational problems would arise?

(Your answer will confirm whether the full picture has clicked. When you're ready afterward, we can do the final wrap-up summary.)

A's eigenvalues can become unstable. Im not sure about computational

Great — you've nailed the **stability** part. Now let's complete the computational side in a clear, slide-grounded way.

✓ Why Updating A Every Step Breaks Computation

If the model were allowed to *change* A at every token instead of keeping A fixed and modulating only Δt , two major computational issues arise:

1. Loss of the convolutional view (lec17 p.5–10)

The entire efficient-training story depends on the identity:

$$y_t = \sum_{\tau} C A^{\tau} B u_{t-\tau},$$

which becomes a **single, fixed convolution kernel**

$$K_{\tau} = C A^{\tau} B.$$

This only works **when A is constant** over time.

If A_t changes at every step:

- The kernel becomes input-dependent and varies at every location.
- There is **no single convolution** that represents the sequence.
- You lose the ability to train in parallel across all time steps.

(This is exactly what the slides emphasize: SSMs work because they have both *recurrent and convolutional* views.)

So training becomes **fully sequential**, just like a vanilla RNN — but even heavier because the recurrence is now a large matrix update, not a simple nonlinearity.

2. Huge computational overhead at inference time

Right now, inference is cheap because we repeatedly apply the *same* matrix exponential:

$$h_{t+1} = e^{A\Delta t(u_t)} h_t.$$

If A_t changes at each step, we would need to compute:

$$e^{A_t \Delta t_t}$$

for every token.

Matrix exponentiation is expensive:

- $O(N^3)$ per step in general
- repeated T times $\rightarrow O(TN^3)$
- completely destroys the linear-time inference advantage emphasized on *lec17 p.10*.

The whole point of the selective-scan mechanism (*lec17 p.10*) is that **input modulation happens through Δt** , which is a *scalar*, not by changing an entire matrix.

3. Additional stability reason (the one you already gave)

Yes — updating A_t each step means its eigenvalues could drift outside the unit circle due to:

- optimizer noise
- floating-point accumulation

- input-dependent gating

which is exactly what *lec16 p.2* warns about.

By freezing A , we guarantee:

- the eigenvalues remain stable,
- the memory spectrum stays predictable,
- training cannot accidentally push the system into unstable dynamics.

✓ Perfect One-Sentence Summary

Fixing A preserves stability and gives us convolutional parallelism, while modulating Δt gives input-dependent expressivity at negligible computational cost.

★ Final Understanding Check

If A is fixed and stable, why is it *safe* for the model to apply large or small values of $\Delta t(u_t)$ without ever causing instability?

(Answer this, and then I'll give you the final wrap-up tying everything together across *lec16 + lec17*.)

im not sure

No problem — this is the *last* conceptual hinge, and once it clicks, the whole SSM story becomes clean and intuitive.

Let's walk through it slowly, grounded in the CT→DT equations from *lec17 p.5–7*.

✓ Why Modulating Δt Cannot Cause Instability

Recall the discrete-time update for a fixed continuous-time A :

$$h_{t+1} = e^{A \Delta t(u_t)} h_t + \bar{B}(u_t) u_t.$$

The key term is the matrix exponential $e^{A \Delta t}$.

Fact (from CT→DT theory):

If the continuous-time matrix A has eigenvalues with **negative real part**, then

$$e^{A \Delta t}$$

has eigenvalues strictly **inside the unit circle** for **any** $\Delta t > 0$.

This is exactly what the lecture shows on *lec17 p.6*: the plots illustrate that as Δt increases, the discrete eigenvalues shrink toward 0 — they never cross outside the unit circle.

✓ Why This Guarantees Stability

Because the eigenvalues of A determine stability in CT, and because A is fixed and chosen to be stable (*lec16 p.2–3*):

- Δt can be tiny → eigenvalues close to 1 → long memory
- Δt can be huge → eigenvalues shrink toward 0 → fast forgetting
- **But** eigenvalues never jump outside the unit circle → **never explode**

So:

Changing Δt only moves discrete eigenvalues *along stable trajectories*.

This is why selective gating is safe:

you can dynamically stretch or compress memory *without ever destabilizing the system*.

✓ Short Intuitive Version

- A defines “safe” continuous-time dynamics.
- Exponentiating a stable CT system always yields a stable DT system.
- Therefore, Δt cannot make the system unstable — it only changes the *speed* of decay.

This is one of the core mathematical reasons selective SSMs work.

★ Final Check

Now that you've seen the explanation, try a short reformulation:

In your own words: why does keeping A fixed but modulating $\Delta t(u_t)$ ensure both stability *and* dynamic flexibility?

(After this, I'll give you the full final wrap-up summarizing the entire SSM storyline.)

since A is fixed and the real parts of A 's eigenvalues are negative and < 1 , no matter what Δt we choose, $e^{\Delta t} * \lambda$ will always be between 0 and 1 thus ensuring stability. by modulating Δt at every input, we can change the memory giving us dynamic flexibility

Beautiful — that is exactly the correct understanding, and it matches the CT→DT stability reasoning on *lec17 p.6–7* and the eigenvalue stability diagrams on *lec16 p.2*. You've now internalized the core mathematical structure that makes modern SSMs (like Mamba) both expressive **and** stable.

As promised, here is your **full wrap-up**, integrating everything from *lec16 + lec17* into a tight final storyline.

★ Final Wrap-Up: The Whole SSM Story (*lec16 + lec17*)

1. Stability & Eigenvalues → Memory Behavior

From *lec16*, the heart of SSMs lies in the eigenvalues of A :

- $|\lambda| < 1 \rightarrow$ memory decays
- $|\lambda| > 1 \rightarrow$ memory explodes

- $|\lambda| \approx 1 \rightarrow$ long-range memory

This is why SSMs use **structured initialization** (e.g., HIPPO) and often **complex eigenvalues** (lec16 p.3): they produce expressive oscillatory modes while keeping all eigenvalues *just inside* the unit circle for stability.

Key insight:

The memory spectrum is encoded in the eigenvalues of A .

2. CT $\rightarrow$ DT Conversion $\rightarrow$ Practical SSM Layers

From *lec17 p.5–7*, SSMs are most naturally understood in continuous time:

$$\dot{x}(t) = Ax(t) + Bu(t).$$

Discretizing with step size Δt :

$$x_{t+1} = e^{A\Delta t}x_t + \bar{B}(\Delta t)u_t.$$

Changing Δt changes the **effective** discrete eigenvalues:

$$\lambda_{DT} = e^{\lambda_{CT}\Delta t}.$$

- Small $\Delta t \rightarrow$ eigenvalues near 1 $\rightarrow$ long memory
- Large $\Delta t \rightarrow$ eigenvalues shrink $\rightarrow$ fast forgetting

This one scalar Δt controls the entire memory spectrum **without ever breaking stability**, because exponentiating a stable CT matrix always yields a stable DT matrix.

3. Selective Gating $\rightarrow$ Adaptive Long-Range Reasoning

On *lec17 p.7–10*, the slides show the key innovation of Mamba:

$$\Delta t = f(u_t) \quad (\text{input-dependent}).$$

This means every token chooses **how fast the system should evolve**:

- Important tokens $\rightarrow$ small $\Delta t \rightarrow$ retain context
- Irrelevant/noisy tokens $\rightarrow$ large $\Delta t \rightarrow$ forget quickly

This is far more expressive than standard RNN gates because the modulation happens through the **matrix exponential** $e^{A\Delta t}$, which scales **all dynamical modes** (fast, slow, oscillatory) in a structured way.

Thus:

- A provides stable, multi-timescale memory physics.
- $\Delta t(u_t)$ provides dynamic, token-level control over those timescales.

4. Unified View: Recurrence + Convolution + Stability + Gating

The entire SSM framework unifies four perspectives:

Recurrence (inference)

$$h_{t+1} = e^{A\Delta t(u_t)} h_t + \bar{B}(u_t) u_t.$$

Linear-time, state-based, efficient for long sequences.

Convolution (training)

$$y_t = \sum_{\tau} C A^{\tau} B u_{t-\tau}.$$

Parallelizable, compatible with FFT or fast kernel computation.

Stability (lec16)

Eigenvalues of A chosen to lie inside the unit circle $\rightarrow$ reliable, non-exploding memory.

Adaptive gating (lec17)

$\Delta t(u_t)$ modifies memory depth dynamically, safely, and efficiently.

This is why modern SSMs can compete with attention for long-range reasoning while maintaining linear-time inference.



You've completed the full interactive SSM tutorial.

If you'd like, I can create:

- a **condensed cheat sheet**,
- a **problem set** to test understanding,
- a **diagram-based summary**,
- or a **research-level comparison** between Mamba and other architectures.

What would you like next?